

Lab 4: Memory
Geavanceerde Computerarchitectuur: Labosessie

(B-KUL-JLI1QS)

Timon Coen

March 17, 2024



Group: MEIswe

Instructor: Alex Marinšek

Introduction

In this lab session, the focus lied on memory types and memory access. When accessing memory, the adjacent memory places are also copied to a fast cache. So using this to our advantage, can have a huge performance benefit. Next to the way we access memory, we also looked at different types of memory. There is global memory, shared memory and constant memory. All of these are compared in an example of matrix multiplication.

Coalesced memory

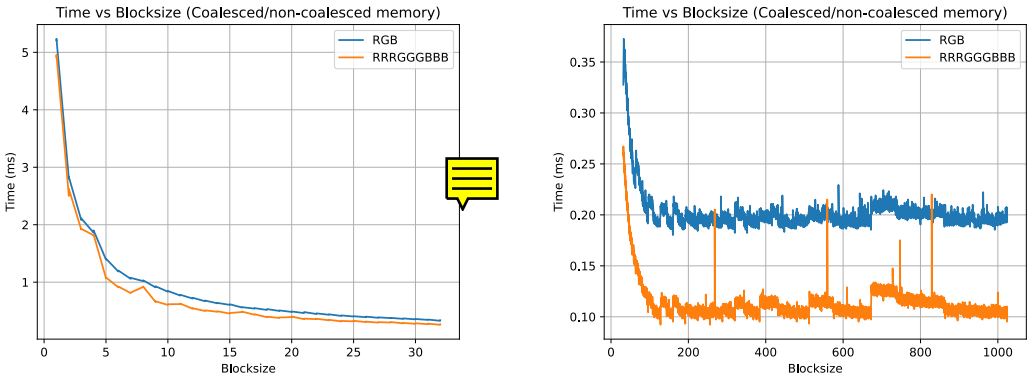
Like already specified in the introduction, when reading form memory, the adjacent memory places are also copied, so reading from them is a far cheaper operation. This in combination with striding also gives a reduced time. The exercise in the lab was to invert the red color. As only the red colour needed to be inverted, the R value was the only one that needed access. Here the placement of the color values in the array is of great importance. Two configurations were tested: `RGB...RGB...RGB` and `RRR...RRRGGG...GGGBBB...BBB`. With the first configuration, it is easy to make operations on a single pixel, the second is easy to use when values of a certain colour need to be accessed. A quick note on this, when all values need to be accessed, both can be good as the next memory place of each color will also be available in both configurations, resulting in relatively equal execution times, as can be seen in figure 2a and 2b where the gray scale is made out of the three colours.
In figure 1a and 1b is clearly visible that coalesced memory access is faster. Again with respect to the block size and stride level, we can see spikes both low and high where the blocksize, warp or stream multiprocessor is fully filled. This can be seen that the execution time lowers gradually per warp size and per stream processor size, resulting in smaller spikes every 32 and bigger spikes every 192. All measurements are done on stride level 32 (32 iterations per thread).

Types of memory

Until now we have worked with global memory, off-device high-latency memory access. This is fine for normal tasks where there is a lot of memory that needs to be accessed and is only needed once. If multiple threads need to access the same memory, shared memory can be a better option. This is low-latency on-device memory. The drawback is that there is a very limited size of 48kB, but read and write times are very fast. If practically all threads need to read from the same memory place, constant memory can be very fast. When accessing different memory places, the reads will be executed sequentially, which will be much slower than global or shared memory. This is also visible in figure 3. For large numbers of elements in the matrix the constant memory gets very slow. The execution times for global and shared memory are in the same order of magnitude, with shared memory executing a little bit faster.

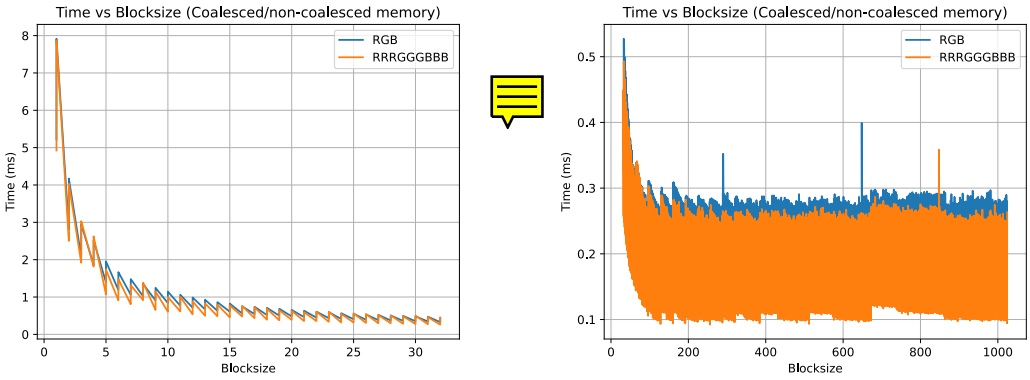
Conclusion

When executing a program on a GPU, the way memory is accessed is crucial. If memory can be placed in adjacent memory places, the execution can be much faster. Then if the same memory needs to be accessed a lot by threads, shared memory can be very useful. If one or a small number of memory places need to be accessed, constant memory can be used. This can be very fast, but only if there is not that much memory access divergence.



(a) (b)

Figure 1: Execution time in ms for coalesced and non-coalesced memory when inverting only the R value



(a) (b)

Figure 2: Execution time in ms for coalesced and non-coalesced memory when making a gray scale

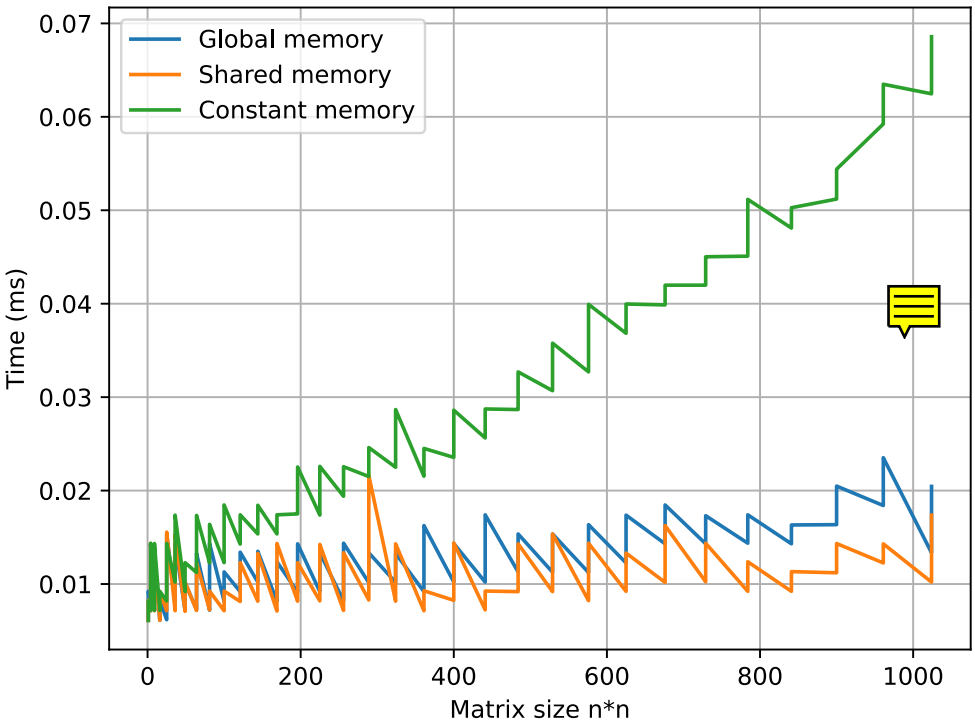


Figure 3: Execution time in ms per matrix size for Global, Shared and Constant memory